



Разбор задачи «Шустрик, Персик и вечная дружба»

Строка состоит из цифр. Разрешены циклические сдвиги влево и вправо. Для любой строки определён потенциал — количество подряд идущих блоков одинаковых символов. Нужно выбрать такой сдвиг, чтобы потенциал был минимален, а число сдвигов — минимально среди всех таких вариантов.

Блоки одинаковых символов

Разобьём строку на максимальные блоки одинаковых символов и запишем её в виде

$$s = c_1^{\ell_1} c_2^{\ell_2} \dots c_k^{\ell_k},$$

где:

- c_i — символ (цифра) в i -м блоке;
- ℓ_i — длина i -го блока, то есть количество подряд идущих символов c_i ;
- $c_i \neq c_{i+1}$ для всех i .

Запись $c_i^{\ell_i}$ означает, что символ c_i повторяется ℓ_i раз подряд. Число блоков k равно потенциалу исходной строки.

Минимальный возможный потенциал

Рассмотрим строку как цикл. Количество мест, где символ меняется, не зависит от сдвига.

- Если $k = 1$, все символы одинаковы, минимальный потенциал равен 1.
- Если $k > 1$ и $c_1 \neq c_k$, в цикле k смен символов, минимальный потенциал равен k .
- Если $k > 1$ и $c_1 = c_k$, крайние блоки в цикле сливаются, смен символов $k - 1$, и минимальный потенциал равен $k - 1$.

Когда сдвиг не нужен

В первых двух случаях ($k = 1$ или $c_1 \neq c_k$) исходная строка уже имеет минимальный потенциал, поэтому ответ равен 0.

Когда нужен сдвиг

Если $k > 1$ и $c_1 = c_k$, нужно начать строку с границы между разными символами. Ближайшие такие границы:

- между первым и вторым блоком — сдвиг влево на ℓ_1 ;
- между предпоследним и последним блоком — сдвиг вправо на ℓ_k .

Меньшее из этих чисел и есть минимальное количество операций.

Разбор задачи «Магический круг»

Упрощение условия

Изначально у всех магов одинаковое количество маны (10^{18}), поэтому для ответа достаточно учитывать **только суммарное изменение маны**. Обозначим через $\Delta[k]$ итоговое изменение маны мага k . Требуется найти k с максимальным $\Delta[k]$ (при равенстве — минимальный номер).

Перейдём к нумерации магов от 0 до $n - 1$. Тогда i -й ритуал затрагивает все позиции вида

$$(first_i - 1 + step_i \cdot x) \bmod n, \quad x \geq 0.$$



Ключевое наблюдение: роль gcd

Пусть

$$g = \gcd(\text{step}_i, n).$$

Из теории чисел следует:

- последовательность $(\text{first}_i - 1 + \text{step}_i \cdot x) \bmod n$ посещает ровно $\frac{n}{g}$ различных позиций;
- это ровно все позиции k , для которых

$$k \equiv (\text{first}_i - 1) \pmod{g}.$$

То есть каждый ритуал добавляет cost_i всем магам из одного класса вычетов по модулю g .

Группировка ритуалов

Для каждого ритуала вычислим пару

$$(g, r), \quad \text{где } g = \gcd(\text{step}_i, n), \quad r \equiv (\text{first}_i - 1) \bmod g.$$

Все ритуалы с одинаковыми (g, r) действуют на один и тот же набор магов, поэтому их эффекты можно суммировать:

$$S_{g,r} = \sum \text{cost}_i.$$

Тогда итоговое изменение маны мага k равно

$$\Delta[k] = \sum_{g|n} S_{g, k \bmod g}.$$

Как восстановить Δ

Для каждой пары (g, r) с $S_{g,r} \neq 0$ нужно добавить $S_{g,r}$ всем индексам

$$r, r + g, r + 2g, \dots \quad (\text{по модулю } n),$$

то есть $\frac{n}{g}$ позициям.

Оценка сложности

Для фиксированного g суммарная работа по всем остаткам r оценивается так:

$$\sum_{r=0}^{g-1} \frac{n}{g} = n.$$

Значит общая сложность равна

$$O(n \cdot d(n)),$$

где $d(n)$ — число делителей n .

Используем стандартную оценку:

$$d(n) \leq 2\sqrt[3]{n}.$$

Тогда

$$n \cdot d(n) \leq 2n\sqrt[3]{n}.$$

При $n \leq 2 \cdot 10^5$ это порядка 10^7 операций, что уверенно укладывается в ограничения.



Подзадачи и путь к полному решению

Группы 1–3: $n, q \leq 5000$

Можно напрямую симулировать каждый ритуал: начать с позиции $first - 1$ и идти шагом $step$, пока не вернёмся в стартовую точку. Длина цикла равна $\frac{n}{\gcd(step, n)}$, что при малых n допустимо.

Группы 4–6: $step_i \mid n$

Если $step \mid n$, то $\gcd(step, n) = step$. Маги разбиваются на $step$ независимых классов по остатку $k \bmod step$. Каждый ритуал воздействует ровно на один такой класс. Отсюда естественно возникает идея суммировать эффекты по $(step, first \bmod step)$ и затем раздать их всем магам этого класса.

Группы 7–9: $step_i \leq n$

Здесь видно, что важен не сам $step$, а величина $\gcd(step, n)$. Даже если $step$ не делит n , ритуал всё равно проходит по одному классу вычетов по модулю g . Это приводит к обобщению решения с заменой $step \rightarrow \gcd(step, n)$.

Разбор задачи «Приятные пути»

Рассмотрим процесс прогулок Шуршика. Каждая прогулка начинается в корне дерева и заканчивается в некоторой листовой вершине. При этом на каждом шаге выбирается ребро с минимальной протоптанностью, а при равенстве протоптанностей — ведущее в вершину с минимальным номером. После завершения прогулки протоптанность всех рёбер выбранного пути увеличивается на единицу.

Частный случай: дерево-звезда

Сначала разберём упрощённый случай, когда дерево является звездой, то есть все вершины, кроме корня, являются его непосредственными детьми.

В этом случае задачу можно переформулировать следующим образом. Пусть дан массив a длины $n-1$, где a_i — протоптанность ребра от корня к вершине $i+1$. Необходимо q раз выполнить операцию: увеличить на 1 самый левый минимальный элемент массива. Требуется определить, сколько раз будет увеличен каждый элемент массива.

Заметим, что минимальный элемент массива в процессе выполнения операций монотонно не убывает. Следовательно, можно бинарным поиском найти значение mn — минимальный элемент массива после выполнения всех q операций.

Для фиксированного значения mn легко проверить, сколько операций требуется, чтобы все элементы массива стали не меньше mn :

$$S(mn) = \sum_{i=1}^{n-1} \max(0, mn - a_i).$$

Если $S(mn) \leq q$, то достижение такого минимума возможно.

После нахождения итогового значения mn остаётся выполнить

$$q - S(mn)$$

дополнительных операций. Поскольку на этом этапе минимум массива уже равен mn , количество таких операций не превосходит $n-1$, и их можно смоделировать напрямую, каждый раз увеличивая самый левый минимальный элемент.

Отметим, что вместо бинарного поиска можно отсортировать массив и линейно увеличивать минимум, поддерживая значение $S(mn)$, однако этого не требовалось



Обобщение на произвольное дерево

Рассмотрим теперь исходное дерево. Пусть из корня выходит несколько рёбер. Выбор первого ребра каждой прогулки зависит только от их текущих протоптанностей и полностью эквивалентен рассмотренному выше случаю со звездой.

Таким образом, мы можем определить, сколько раз за q прогулок будет использовано каждое ребро, исходящее из корня. Пусть для ребра $(1, v)$ это число равно q_v . После этого задача сводится к аналогичной задаче для поддерева с корнем в вершине v , где количество прогулок равно q_v .

Следовательно, решение строится рекурсивно: выполняя обход дерева, для каждой вершины мы распределяем полученное от родителя количество проходов между её детьми по описанному алгоритму. Значение q , передаваемое в поддерево, равно числу раз, которое было пройдено ребро от родителя к данной вершине.

Когда обход достигает листовой вершины, переданное ей значение q и есть количество раз, которое Шуршик побывал в этом листе.

Сложность

На каждой вершине требуется обработать рёбра, исходящие из неё. Суммарно по всему дереву сортировки и вычисления выполняются за

$$O(n \log n),$$

что укладывается в ограничения задачи.

Разбор задачи «Хомячья зрелищность»

Краткое условие

Для отрезка $[L, R]$ рассмотрим все числа, которые являются делителями хотя бы одного из чисел

$$a_L, a_{L+1}, \dots, a_R.$$

Пусть их количество равно $X(L, R)$. Тогда зрелищность этого отрезка:

$$F(L, R) = X(L, R) - (R - L + 1).$$

Нужно найти

$$\max_{1 \leq L \leq R \leq n} F(L, R).$$

Главная идея: фиксируем правую границу

Будем идти слева направо и фиксировать правую границу R . Попробуем быстро находить

$$\max_{1 \leq L \leq R} F(L, R).$$

Для этого введём важное понятие.

Для каждого делителя d посмотрим на последнюю позицию не правее R , на которой встретилось число, делящееся на d . Обозначим эту позицию через $last_R(d)$.

Тогда делитель d встречается на отрезке $[L, R]$ тогда и только тогда, когда

$$last_R(d) \geq L.$$

Следовательно,

$$X(L, R) = \#\{d \mid last_R(d) \geq L\}.$$

Это уже очень полезная формула: наличие делителя на отрезке определяется только его последним появлением.



Переход к массиву “последних появлений”

Для фиксированного R определим массив cnt :

$$cnt_i = \#\{d \mid last_R(d) = i\}.$$

То есть cnt_i — сколько различных делителей имеют свою последнюю встречу ровно в позиции i .

Тогда количество различных делителей на отрезке $[L, R]$ равно

$$X(L, R) = \sum_{i=L}^R cnt_i,$$

потому что каждый делитель учитывается ровно один раз — в точке своего последнего появления.

Значит,

$$F(L, R) = \sum_{i=L}^R cnt_i - (R - L + 1) = \sum_{i=L}^R (cnt_i - 1).$$

Введём

$$b_i = cnt_i - 1.$$

Тогда

$$F(L, R) = \sum_{i=L}^R b_i.$$

Итак, для фиксированного R задача свелась к следующей:

Нужно найти максимальную сумму суффикса массива $b_1, b_2, \dots, b_R$.

То есть

$$\max_{1 \leq L \leq R} F(L, R) = \max_{1 \leq L \leq R} \sum_{i=L}^R b_i.$$

Как меняется массив b при переходе к следующей позиции

Пусть мы обработали позиции до $R - 1$ и теперь добавляем число a_R .

Рассмотрим любой делитель d числа a_R .

- Если раньше последний раз этот делитель встречался в позиции p , то теперь он перестаёт быть “последним” в позиции p :

$$cnt_p \leftarrow cnt_p - 1.$$

- Теперь его последняя встреча — это позиция R :

$$cnt_R \leftarrow cnt_R + 1.$$

Если у числа a_R всего $\tau(a_R)$ делителей, то:

- в позиции R значение cnt_R увеличивается на $\tau(a_R)$;
- в старых позициях последних появлений соответствующих делителей происходят уменьшения на 1.

Следовательно, в массиве b :

- новая позиция получает значение

$$b_R = \tau(a_R) - 1;$$

- в некоторых старых позициях выполняется

$$b_p \leftarrow b_p - 1.$$



Подзадачи 1–3: прямые решения

Совсем наивное решение

Можно перебрать все отрезки $[L, R]$ и для каждого собирать множество всех делителей чисел на нём. Тогда считаем

$$F(L, R) = |S| - (R - L + 1).$$

Это подходит только для самых маленьких ограничений.

Чуть лучше

Если для каждого числа заранее выписать все его делители, то для фиксированного L можно двигать R вправо и постепенно добавлять делители в структуру данных.

Такое решение уже заметно лучше и проходит несколько маленьких групп, но для больших ограничений всё равно слишком медленное.

Промежуточное решение: дерево отрезков

Из предыдущих наблюдений видно, что при движении правой границы:

- в новой позиции R появляется значение $\tau(a_R) - 1$;
- для некоторых старых позиций выполняются уменьшения на 1.

А нам после этого нужно знать максимальную сумму суффикса массива b .

Это можно поддерживать деревом отрезков, храня в каждой вершине:

- сумму на сегменте;
- максимальную суффиксную сумму на сегменте.

Тогда каждое уменьшение в старой позиции — точечное обновление, добавление новой позиции — ещё одно точечное обновление, а ответ для текущего R лежит в корне.

Такое решение уже проходит средние подзадачи, но в полной задаче множитель $\log n$ становится слишком дорогим, потому что уменьшений очень много.

Ключ к полной задаче

Чтобы избавиться от дерева отрезков, нужно посмотреть на задачу по-другому.

Для фиксированного R введём величины

$$S_L = F(L, R) = \sum_{i=L}^R b_i.$$

То есть S_L — зрелищность отрезка, который заканчивается в R и начинается в L .

Тогда текущий ответ для фиксированного R равен

$$\max_{1 \leq L \leq R} S_L.$$

Теперь разберём, как меняются значения S_L .



Что делает уменьшение в позиции p

Если мы уменьшаем b_p на 1, то это влияет на все суммы S_L , для которых $L \leq p$, потому что позиция p входит именно в такие отрезки.

То есть

$$S_1, S_2, \dots, S_p$$

уменьшаются на 1, а

$$S_{p+1}, S_{p+2}, \dots, S_R$$

не меняются.

Итак, уменьшение в позиции p — это **уменьшение префикса** массива S на 1.

Что делает добавление новой позиции R

Если в новой позиции R мы записываем

$$b_R = x = \tau(a_R) - 1,$$

то каждая старая сумма S_L увеличивается на x , потому что в конец каждого отрезка добавился новый элемент с весом x .

Кроме того, появляется новый отрезок длины 1:

$$S_R = x.$$

Значит:

- все старые S_L увеличиваются на x ;
- в конец добавляется новое значение x .

В частности, максимум тоже увеличивается на x .

Монотонная оболочка максимумов

Теперь рассмотрим массив

$$H_i = \max_{j \geq i} S_j.$$

Это максимум на суффиксе значений S .

Тогда:

- ответ для текущего R равен H_1 ;
- массив H не возрастает:

$$H_1 \geq H_2 \geq \dots \geq H_R.$$

Введём разности:

$$diff_i = H_i - H_{i+1}, \quad H_{R+1} = 0.$$

Тогда все $diff_i \geq 0$, а

$$H_1 = \sum_{i=1}^R diff_i.$$

То есть ответ равен сумме этих неотрицательных разностей.



Что происходит при добавлении новой позиции

Если в конец добавляется значение $x = \tau(a_R) - 1$, то все старые S_L увеличиваются на x . Значит, все старые H_i тоже увеличиваются на x . Новая величина H_R становится равной x .

Отсюда следует очень приятный факт:

Все старые $diff_i$ не меняются, а в конец просто дописывается новое значение

$$diff_R = x.$$

Что происходит при уменьшении префикса $S_1, \dots, S_p$ на 1

Это более тонкий момент. Массив H — это невозрастающая “верхняя оболочка” массива S . Это означает, что левая часть массива S опускается на 1, а правая часть ($S_{p+1}, \dots, S_R$) не меняется.

Теперь посмотрим, как изменится массив максимумов H .

Поскольку $H_i = \max(S_i, S_{i+1}, \dots, S_R)$, то для $i > p$ ничего не изменится, потому что все соответствующие S остались прежними.

А для $i \leq p$ максимум либо был достигнут справа (и тогда он не изменится), либо был достигнут в левой части (и тогда он уменьшится на 1).

Граница между этими двумя случаями проходит по уровню H_{p+1} :

- все значения H_i , которые были равны H_{p+1} , не изменятся;
- все значения H_i , которые были строго больше H_{p+1} , уменьшатся на 1.

Если посмотреть на разности

$$diff_i = H_i - H_{i+1},$$

то уменьшение этого “верхнего куска” означает, что уменьшается не более одного значения $diff_i$ — самое правое положительное $diff_i$ среди позиций $i \leq p$.

Это и есть ключевая экономия: вместо обновления целого префикса мы меняем только **один** элемент массива $diff$.

Почему этого достаточно

Мы пришли к очень простой динамике.

При обработке очередного a_R :

1. для каждого делителя d числа a_R :
 - если раньше d последний раз встречался в позиции p , то нужно уменьшить на 1 самое правое положительное $diff_i$ среди $i \leq p$;
 - затем записать, что теперь последняя встреча делителя d — это позиция R ;
2. после обработки всех делителей дописать в конец

$$diff_R = \tau(a_R) - 1;$$

3. сумма всех текущих $diff_i$ и есть лучший ответ среди всех отрезков, оканчивающихся в R .

Остаётся только уметь быстро делать две операции:

- дописать в конец новое неотрицательное число;
- по числу p найти самое правое положительное $diff_i$ среди $i \leq p$ и уменьшить его на 1.



Как это поддерживать быстро

Будем хранить только позиции, в которых

$$diff_i > 0.$$

Если некоторое $diff_i$ стало равно нулю, такая позиция больше не нужна.

Нужно уметь:

- находить предыдущую “живую” позицию;
- удалять позицию, если её значение обнулилось;
- добавлять новую позицию в конец.

Это можно сделать структурой “следующий/предыдущий живой элемент”: например, через DSU со сжатием путей и двусвязный список.

Тогда каждое уменьшение делается почти за $O(1)$ амортизированно: мы сразу прыгаем к нужной живой позиции и уменьшаем её значение. Если оно стало нулём, удаляем позицию из структуры.

Факторизация и перечисление делителей

Чтобы для каждого числа a_i быстро находить все его делители, заранее считаем массив минимальных простых делителей для всех чисел до максимального значения.

Тогда каждое число быстро раскладывается на простые множители, после чего по этому разложению рекурсивно перечисляются все его делители.

Для каждого делителя мы обновляем его последнюю позицию появления.

Разбор задачи «Bracket Dance»

Общие наблюдения

Решим сначала задачу подсчёта числа различных перестановок.

Заметим, что для разных x блоки, в которых мы меняем местами их левую и правую половины, вложены друг в друга. На самом деле для определённой скобки по набору действий мы можем понять, где она окажется. А именно — сколько раз она переходила из левой половины строки в правую и обратно (то есть меняла чётность половины строки, в которой находится), сколько раз меняла чётность четверти и так далее. Видно что для итоговой позиции скобки неважен порядок действий, а также важна только чётность числа действий для каждого x .

Всего $\log_2 N$ типов действий.

Переберём все конфигурации (какие действия выполнены нечётное число раз, какие нет) — их $2^{\log_2 N} = N$.

Теперь для каждой конфигурации хотим понять, верно ли что итоговая строка — ПСП.

Способ 1

Перебираем N конфигураций, для каждой строим строку заново и проверяем, является ли она ПСП.

Построение строки будет занимать не более $N \log_2 N$ — применяем $\log_2 N$ шагов к строке, на каждом шаге перестраиваем её за длину строки.

Получим решение за $\mathcal{O}(N^2 \log N)$.

Способ 2

Если применена операция x — скобка перемещается в соседний блок длины 2^{x-1} , причём с точки зрения блоков большей длины эта скобка осталась там же, а с точки зрения меньших — сохранила относительное положение внутри блока 2^{x-1} .

На самом деле единственный бит позиции (при записи в двоичной системе счисления), который поменяется, это бит под номером $x - 1$. Он поменяет своё значение на противоположное.

Значит применяя несколько операций, если xor их длин равен y , то итоговая строка t представима в таком виде:



$t_i \text{ XOR } y = s_i$ (так как символы переместились с позиции i на позицию $i \text{ XOR } y$)

Значит для текущей конфигурации есть быстрый способ получить строку t . Это даёт решение за $\mathcal{O}(N^2)$.

Способ 3

ПСП — такая строка, что суммарный баланс в ней равен нулю, при этом баланс любого префикса неотрицателен. То есть минимальный баланс по всем префиксам неотрицателен.

Если у нас есть две строки t_1, t_2 , и мы знаем их суммарные балансы b_1, b_2 , а также минимальный баланс по всем префиксам $\min b_1, \min b_2$, то мы умеем пересчитывать эти значения для конкатенации двух строк.

А именно у строки $T = t_1 + t_2$ суммарный баланс $B = b_1 + b_2$, минимальный баланс достигается либо в левой части — $\min b_1$, либо в правой — $b_1 + \min b_2$, а значит равен $\min(\min b_1, b_1 + \min b_2)$.

Итак, осталось придумать хороший способ перебора всех возможных конфигураций с поддержанием списка текущего набора блоков и информации для них.

Заметим, что самая долгая операция объединения — для $x = 1$, когда приходится объединять много маленьких строк, будет много пересчёта балансов.

И наоборот самая быстрая операция объединения — для $x = \log_2 N$, когда объединить нужно всего два отрезка.

Хорошо, тогда если у нас есть дерево рекурсии с перебором конфигураций — стоит у его корня перебирать более тяжёлые операции, и у листов — лёгкие (тогда тяжёлые операции выполняются редко).

Запускаем рекурсию, на очередном слое рекурсии поддерживаем список блоков (для каждого блока — суммарный и минимальный баланс в нём), на очередном слое число блоков будет сокращаться вдвое. В первую очередь перебираем выполнена ли операция для $x = 1$, затем для $x = 2$ и так далее. Получим решение за $\mathcal{O}(N \log N)$ — на слое рекурсии номер i оказываемся 2^i раз, и каждый раз пересчитываем баланс для $\frac{N}{2^i}$ блоков, итого N действий для $\log_2 N$ слоёв рекурсии.

Число различных строк

Для подсчёта числа различных итоговых строк можем все итоговые строки ПСП отсортировать и посчитать число различных. В качестве альтернативы возможно использование сета или хеш-сета. Такой способ подсчёта не является оптимальным.

Для полного решения возможно воспользоваться хешированием, или же идеальным хешированием — все возможные строки длины 2^k пронумеровать. А именно — будем при пересчёте для блока кроме балансов дополнительно хранить номер типа этого блока. При конкатенации двух блоков номер итогового будем задавать по паре (a, b) , где a, b — номера блоков-половин. Тогда будет достаточно посчитать число различных типов итоговых строк ПСП.

Возможно упростить решение и подсчитать только число наборов операций, переводящих из начальную строку в саму себя. Тогда для получения ответа достаточно будет поделить число достижимых перестановок, которые задают ПСП на число наборов операций, переводящих строку в себя.